Esercitazione di Programmazione in C

Gestione della Memoria e Puntatori

Esercizio 1. (Swap) Scrivi una funzione `swap` che accetti due puntatori a interi e scambi i valori delle variabili a cui puntano. Testa la funzione nel `main` richiedendo all'utente due numeri tramite `scanf` e stampandoli prima e dopo lo scambio tramite `printf`.

Esercizio 2. (Stampa di un array) Dichiara staticamente un array di 5 interi. Inizializza un puntatore all'array. Utilizza esclusivamente il puntatore in un ciclo `for` per stampare a schermo tutti gli elementi.

Esercizio 3. (Ricerca del massimo) Scrivi una funzione che riceva un puntatore al primo elemento di un array di interi e un intero che ne rappresenti la dimensione. La funzione deve scorrere l'array tramite l'aritmetica dei puntatori, trovare il valore massimo e restituirlo.

Esercizio 4. (Somma degli elementi) Dichiara nel `main` un array statico di interi. Scrivi una funzione che riceva come parametri un puntatore al primo elemento di questo array e la sua dimensione. La funzione deve scorrere l'array usando esclusivamente dell'aritmetica dei puntatori (senza usare la notazione con le parentesi quadre arr[i]) per calcolare e restituire la somma di tutti i valori. Stampa il risultato nel `main`.

Esercizio 5. (Allocazione dinamica base) Chiedi all'utente quanti numeri desidera inserire. Alloca dinamicamente la memoria necessaria per contenerli usando `malloc()`. Leggi i numeri uno ad uno, calcolane la somma, stampala e ricordati di liberare rigorosamente la memoria al termine tramite `free()`.

Esercizio 6. (Copia di un array) Dichiara nel `main` due array statici di interi della stessa dimensione, inizializzando con dei valori solo il primo. Utilizzando due puntatori, uno che punta all'array sorgente e uno all'array di destinazione, scrivi un ciclo (senza usare la notazione con le parentesi quadre) che copi riga per riga tutti i valori dal primo al secondo array. Al termine, stampa il secondo array per verificare la correttezza della copia.

Esercizio 7. (Clonazione dinamica) Scrivi una funzione che riceva in input un puntatore a un array di interi e la sua dimensione. La funzione deve allocare dinamicamente un nuovo blocco di memoria della stessa dimensione, copiare i valori dall'array originale al nuovo usando l'aritmetica dei puntatori e, infine, restituire il puntatore alla nuova area allocata. Nel `main`, gestisci la chiamata e la successiva `free()`.

Esercizio 8. (Generazione array dinamico) Chiedi all'utente un intero N. Scrivi una funzione che riceva N come parametro, allochi dinamicamente (tramite `malloc()`) un array di N interi e lo riempia con i primi N numeri pari (partendo da 0) usando esclusivamente

l'aritmetica dei puntatori. La funzione deve restituire il puntatore. Nel `main`, recupera l'array, stampalo a schermo e libera la memoria.